

Take-Home Assignment

Data Engineering

Pipeline de Movimientos Financieros

Tiempo	3 días
Stack mínimo	Python · Docker · DuckDB o PostgreSQL
Entregable	Repositorio Git + README

Contexto

En Tyba procesamos diariamente información de movimientos financieros de nuestros clientes. Estos datos se reciben mediante un corte diario: cada día recibimos un archivo con el estado de los movimientos. De un día a otro, los datos presentan cambios dinámicos, aparecen registros nuevos, se corrigen algunos, otros desaparecen, y no siempre llegan limpios.

El objetivo es evaluar el diseño y la construcción de un pipeline de ingeniería de datos de extremo a extremo: que sea confiable, que escale y que produzca datos en los que se pueda confiar para tomar decisiones.

Insumos

- `movimientos_dia_T.parquet` — el corte de un día.
- `movimientos_dia_T1.parquet` — el corte del día siguiente.

Los archivos se proveen junto con este documento. Tu objetivo es construir un pipeline que los ingiera y mantenga una base de datos consultable y consistente a lo largo del tiempo.

¿Qué cambia entre un corte y el siguiente?

Los cortes diarios no son independientes: el del día siguiente es una evolución del anterior. Entre T y T+1 pueden ocurrir estas situaciones, el pipeline debe procesarlas correctamente:

Situación	Descripción
Registro nuevo	Un id que no existía en T aparece en T+1.
Registro corregido	Un id que existía en T aparece en T+1 con distinto amount, description u otro campo.
Registro eliminado	Un id que existía en T no aparece en T+1 (cancelación o corrección de error).
Sin cambios	Un id aparece igual en ambos cortes.

Tu pipeline debe consolidar esa evolución de forma correcta, sin perder trazabilidad ni duplicar información.

 **Tip:** Asume que esto correrá todos los días y que el volumen puede crecer a millones de filas. Lo que funciona con un archivo pequeño en memoria puede no funcionar a escala.

Calidad de los datos

Los archivos pueden contener inconsistencias. Parte del ejercicio es que las identifique, las documentes y decidas cómo manejarlas. No hay una respuesta única correcta, lo que evaluamos es tu criterio y que justifiques las decisiones que tomes.

Glosario de datos

[movimientos_dia_T.parquet](#) / [movimientos_dia_T1.parquet](#)

Campo	Descripción
id	Identificador de la transacción
date	Fecha del movimiento
product	Producto asociado al movimiento
amount	Monto del movimiento
description	Información del movimiento
fund	Fondo en el que se realizó la inversión
type	Si el movimiento es de entrada o de salida
commercial_name	Nombre comercial asociado

Consideraciones técnicas

- La base de datos destino puede ser DuckDB o PostgreSQL.
- El pipeline debe correr con docker compose up --build sin configuración adicional.
- Las decisiones restantes sobre el lenguaje, librerías y arquitectura son de tu elección.
- Diseña pensando en escala: tu solución debe funcionar si el volumen crece a millones de filas.

Estructura de carpetas sugerida (no obligatoria):

```
repo/
├── data/
│   ├── raw/           ← parquet entregados por Tyba
│   └── output/        ← base de datos destino
├── src/
│   └── pipeline.py    ← script principal
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

Entregables

Entregable	Descripción
Repositorio Git	Sube tu solución a GitHub, GitLab o Bitbucket y comparte el enlace.
Dockerfile + docker-compose.yml	El pipeline debe correr con docker compose up --build sin configuración adicional.
Pipeline principal	El código del ETL. Puede estar modularizado en varios archivos.
Insights y analítica	Output en el formato deseado explicando los insights encontrados
README.md	Instrucciones para correr el proyecto.

¡Mucho éxito! Esperamos ver tu solución.